

结构型模式 (OOP)

黄民烈

aihuang@tsinghua.edu.cn

<https://coai.cs.tsinghua.edu.cn/hml>

课程团队：刘知远 任炬 黄民烈

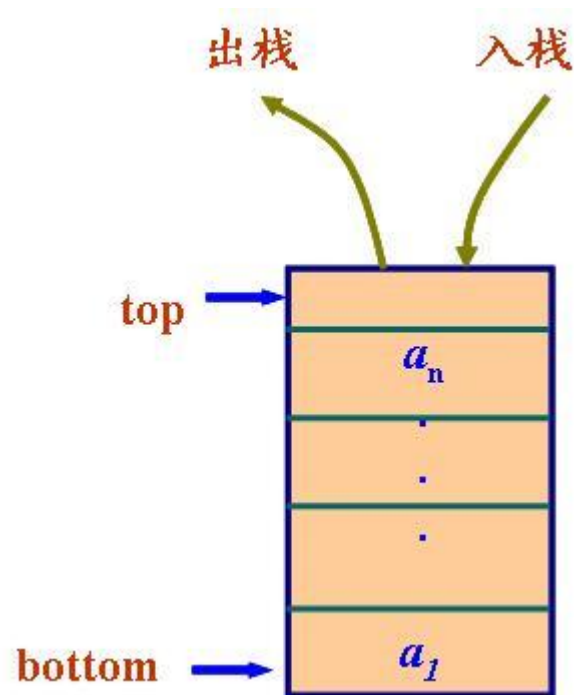
本讲内容提要

- 设计模式：结构型模式
- 13.1 适配器（Adapter）模式
- 13.2 代理/委托（Proxy）模式
- 13.3 装饰器（Decorator）模式

结构型模式

一个简单例子——栈

- 功能类似数组
- 元素访问规则有所不同，是“后进先出”
(Last-In-First-Out)
- 简单起见，只支持int类型的元素



代码实现

```
#include <climits>
#include <vector>
#include <iostream>
using namespace std;

//堆栈基类
class Stack {
public:
    virtual ~Stack() { }
    virtual bool full() = 0;
    virtual bool empty() = 0;
    virtual void push(int i) = 0;
    virtual void pop() = 0;
    virtual int size() = 0;
    virtual int top() = 0;
};
```

简单实现

```
class MyStack : public Stack{

private:
    int *m_data; const int m_size; int m_top;

public:
    //构造函数
    MyStack(int size) : m_size(size), m_top(-1), m_data(NULL) {
        if (m_size > 0) m_data = new int[m_size];
    }
    //析构函数
    virtual ~MyStack() {
        if (m_data) delete [] m_data;
    }
    //满栈检测
    bool full() {
        return m_size <= 0 || (m_top+1) == m_size;
    }
}
```

简单实现

//空栈检测

```
bool empty() {  
    return m_top < 0;  
}
```

//入栈

```
void push(int i) {  
    if (m_top+1 < m_size) m_data[++ m_top] = i;  
}
```

//出栈

```
void pop() { if (!empty()) --m_top; }
```

//获取堆栈已用空间

```
int size() { return m_top+1; }
```

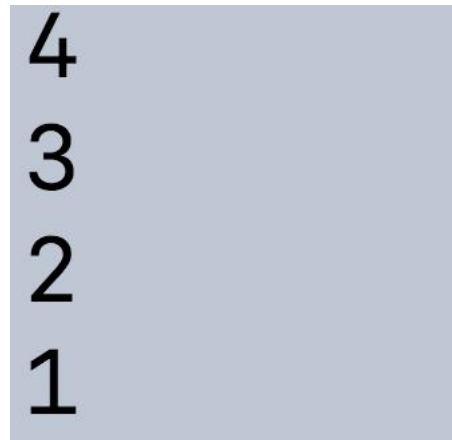
//获取栈头内容

```
int top() {  
    if (!empty())  
        return m_data[m_top];  
    else  
        return INT_MIN;  
}
```

```
};
```

简单实现

```
int main(int argc, char *argv[]) {  
  
    // 创建一个最多放置10个元素的栈  
    MyStack stack(10);  
  
    // 压入1,2,3,4  
    for (int i = 1; i < 5; i++)  
        stack.push(i);  
  
    // 逐个弹出  
    for (int i = 0; i < 4; i++) {  
        cout << stack.top() << "\n";  
        stack.pop();  
    }  
  
    return 0;  
}
```

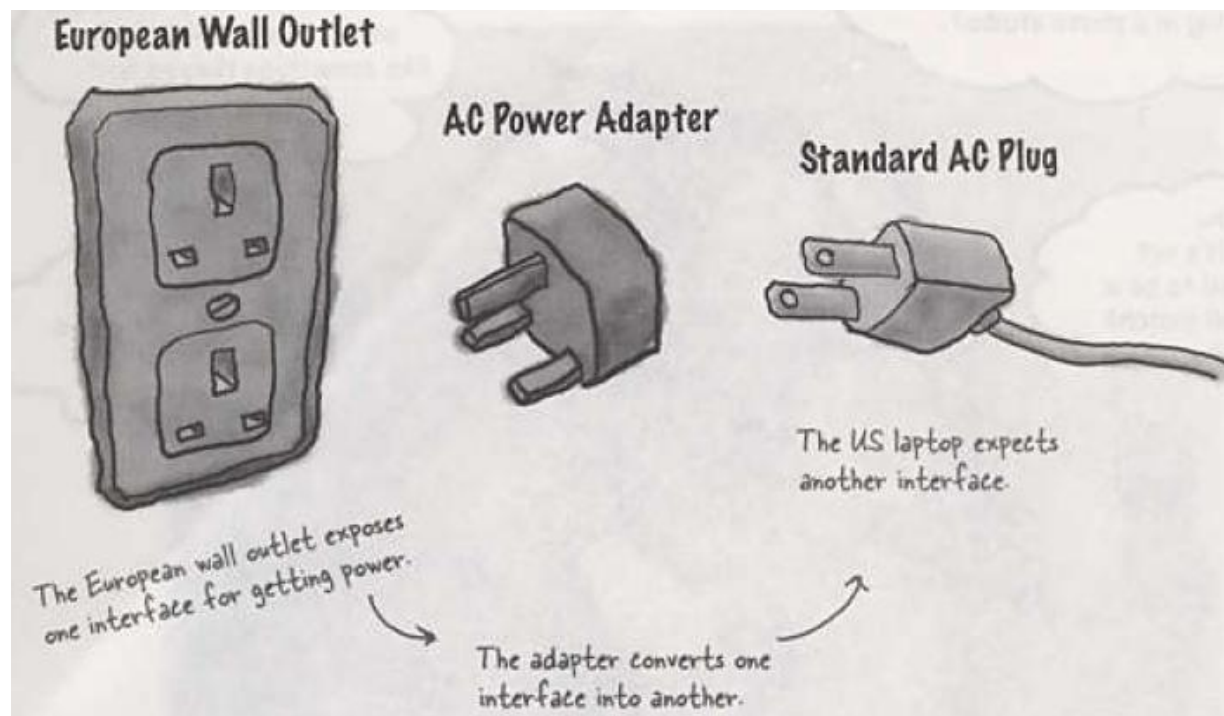


有没有更简单的实现方式?

STL vector

- 工作量太大(**OOP思想之一：复用**)
- 需要我们自行管理内存(**容易出错**)
- 事实上**STL**中有**vector**这个容器
- **vector**提供了如下方法：
 - `push_back()`
 - `size()`
 - `back()`
 - `pop_back()`

分析



■ Vector

- 功能上满足要求（内存管理，元素插入弹出）
- 但是接口不一致

■ 需要进行接口的“转换”

适配器

■考虑生活中一种常见的情况：

- 有手机、手机充电线，要给手机充电。
- 充电线只能插在USB接口上进行充电。
- 但是现在只有220V的插座可以供电。
- 所以需要用一个转接头将220V插座和USB口衔接。

■这里的转接头实际上就是一种现实中的适配器

适配器 Adapter

适配器

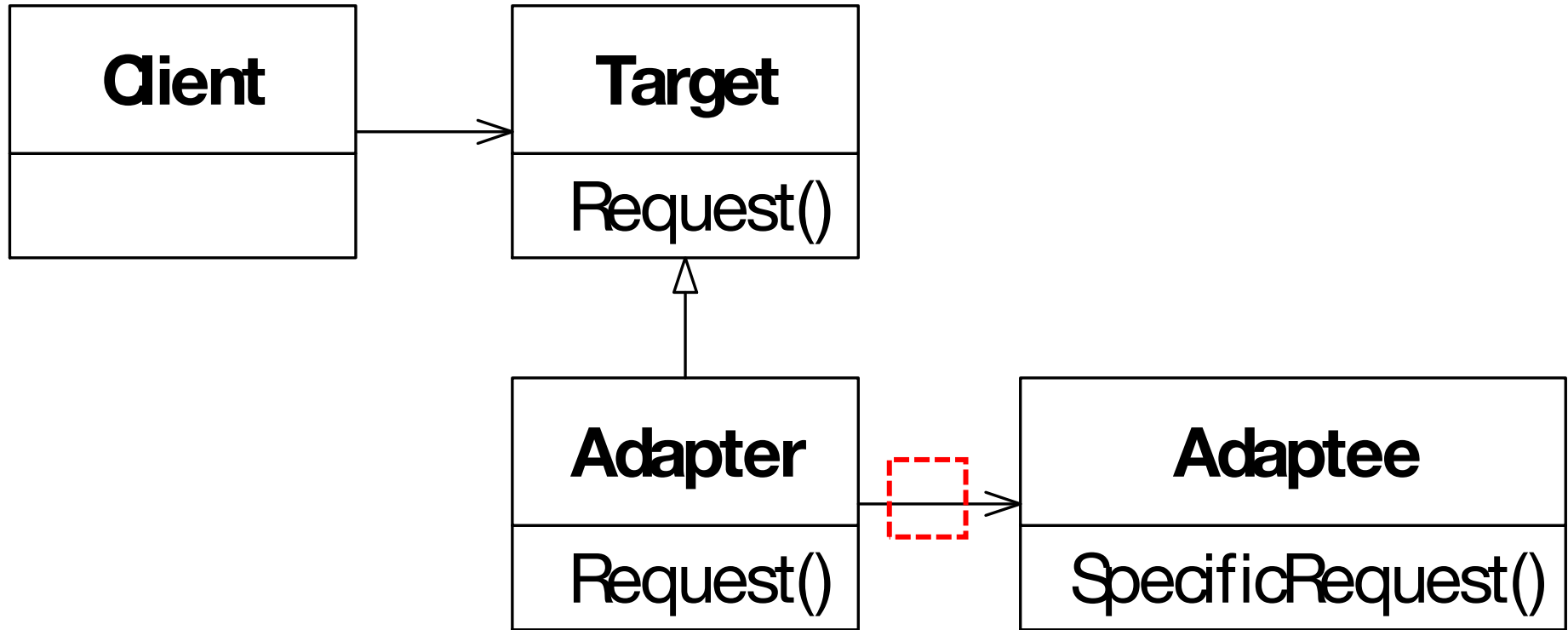
■ 概述

- 适配器模式将一个类的接口转换成客户希望的另一个接口，从而使得原本由于接口不兼容而不能一起工作的类可以在统一的接口环境下工作。

■ 结构

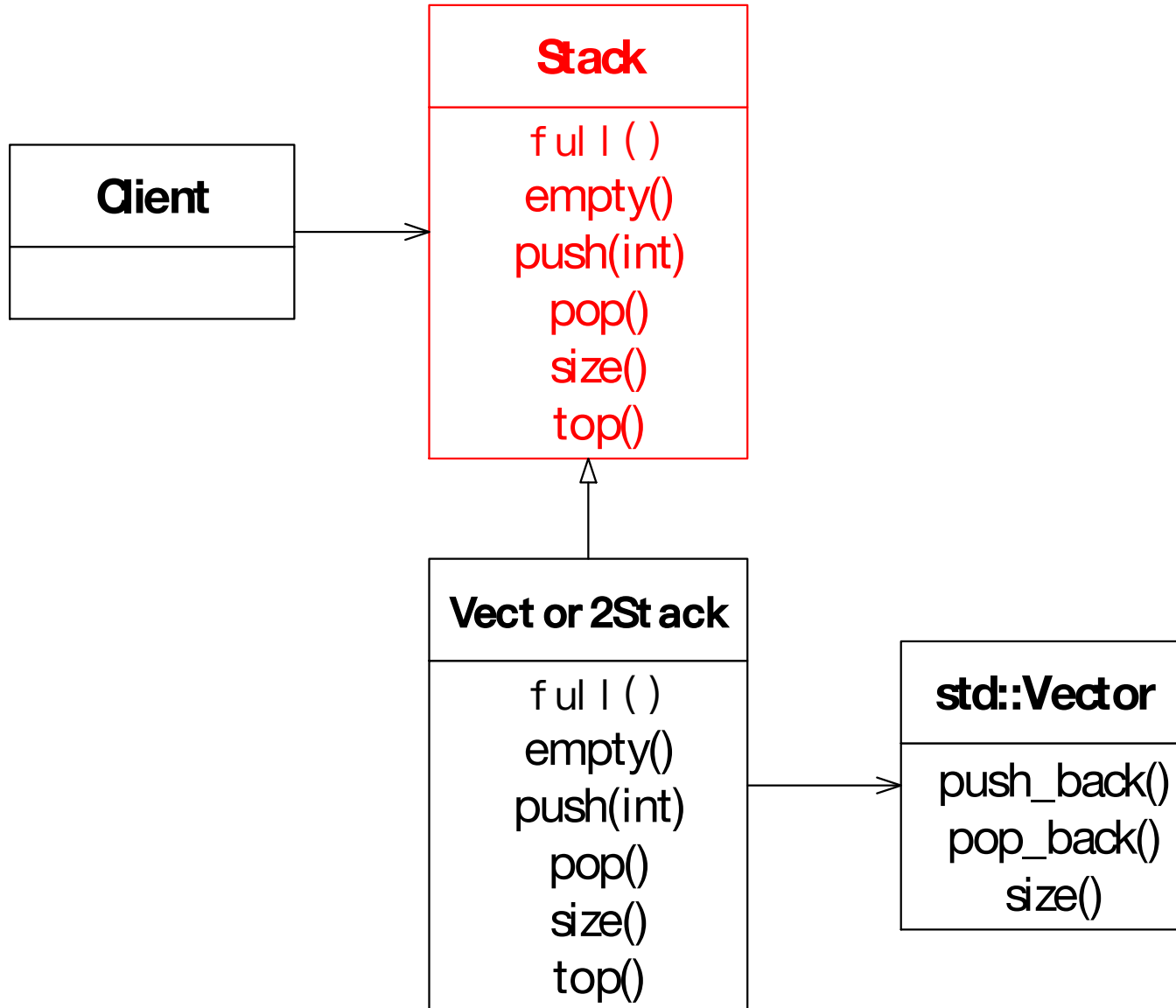
- 目标（Target）：客户所期待的接口。
- 需要适配的类（Adaptee）：需要适配的类。
- 适配器（Adapter）：通过包装一个需要适配的类，把原接口转换成目标接口。

适配器——实现一



使用组合实现适配，称作对象适配器模式

适配器基类定义



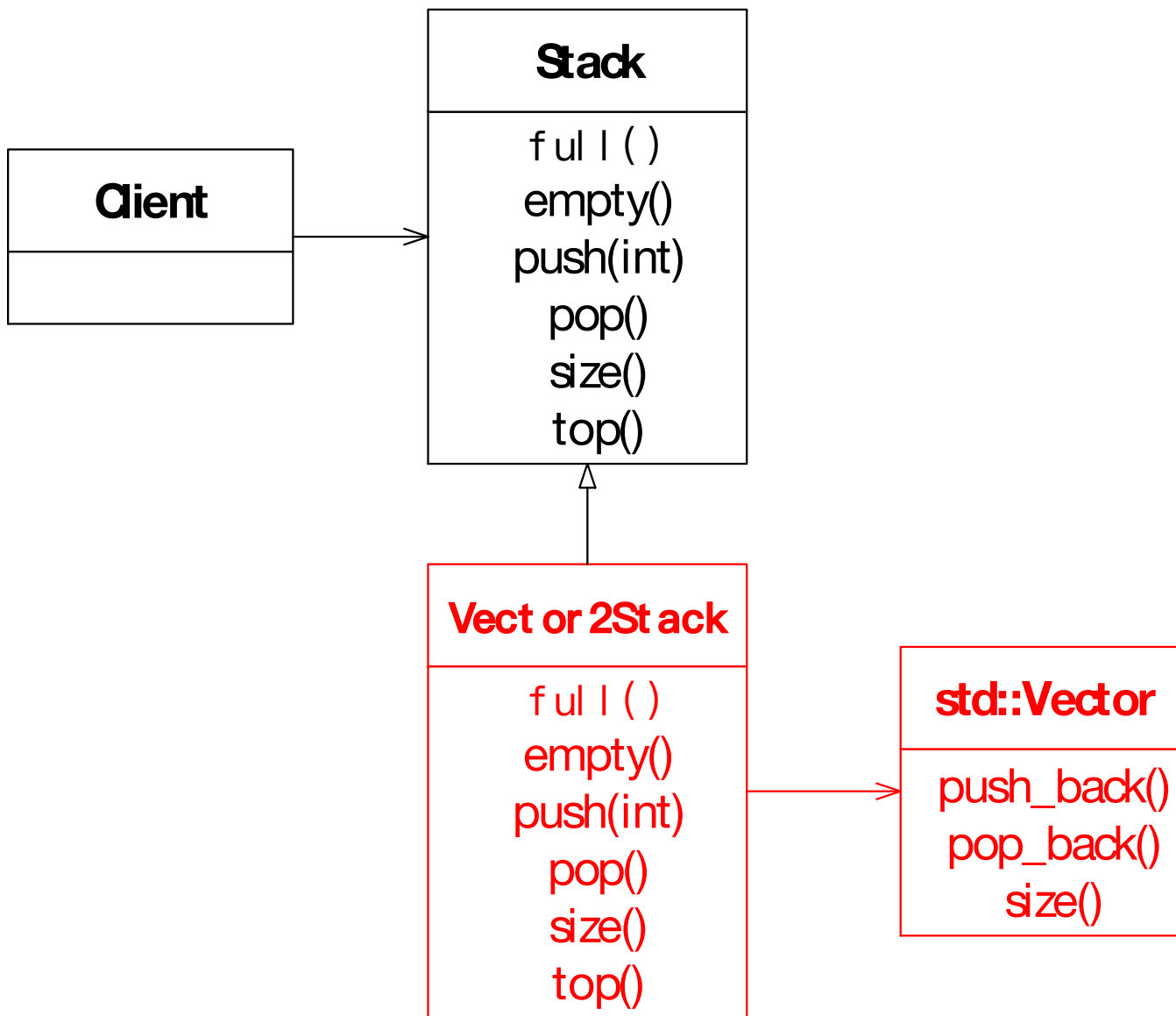
适配器——实现一

```
#include <cstring>
#include <cstdio>
#include <climits>
#include <iostream>
```

//堆栈基类

```
class Stack {
public:
    virtual ~Stack() { }
    virtual bool full() = 0;
    virtual bool empty() = 0;
    virtual void push(int i) = 0;
    virtual void pop() = 0;
    virtual int size() = 0;
    virtual int top() = 0;
};
```

组合方式实现适配器模式

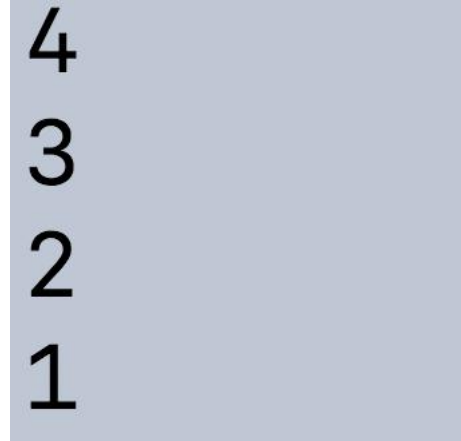


适配器——实现一

```
class Vector2Stack : public Stack{
private:
    std::vector<int> m_data; //将vector的接口组合进来实现具体功能
    const int m_size;
public:
    Vector2Stack(int size) : m_size(size) { }
    bool full() { return (int)m_data.size() >= m_size; } //满栈检测
    bool empty() { return (int)m_data.size() == 0; } //空栈检测
    void push(int i) { m_data.push_back(i); } //入栈
    void pop() { if (!empty()) m_data.pop_back(); } //出栈
    int size() { return m_data.size(); } //获取堆栈已用空间
    int top() { //获取栈头内容
        if (!empty())
            return m_data[m_data.size()-1];
        else
            return INT_MIN;
    }
};
```

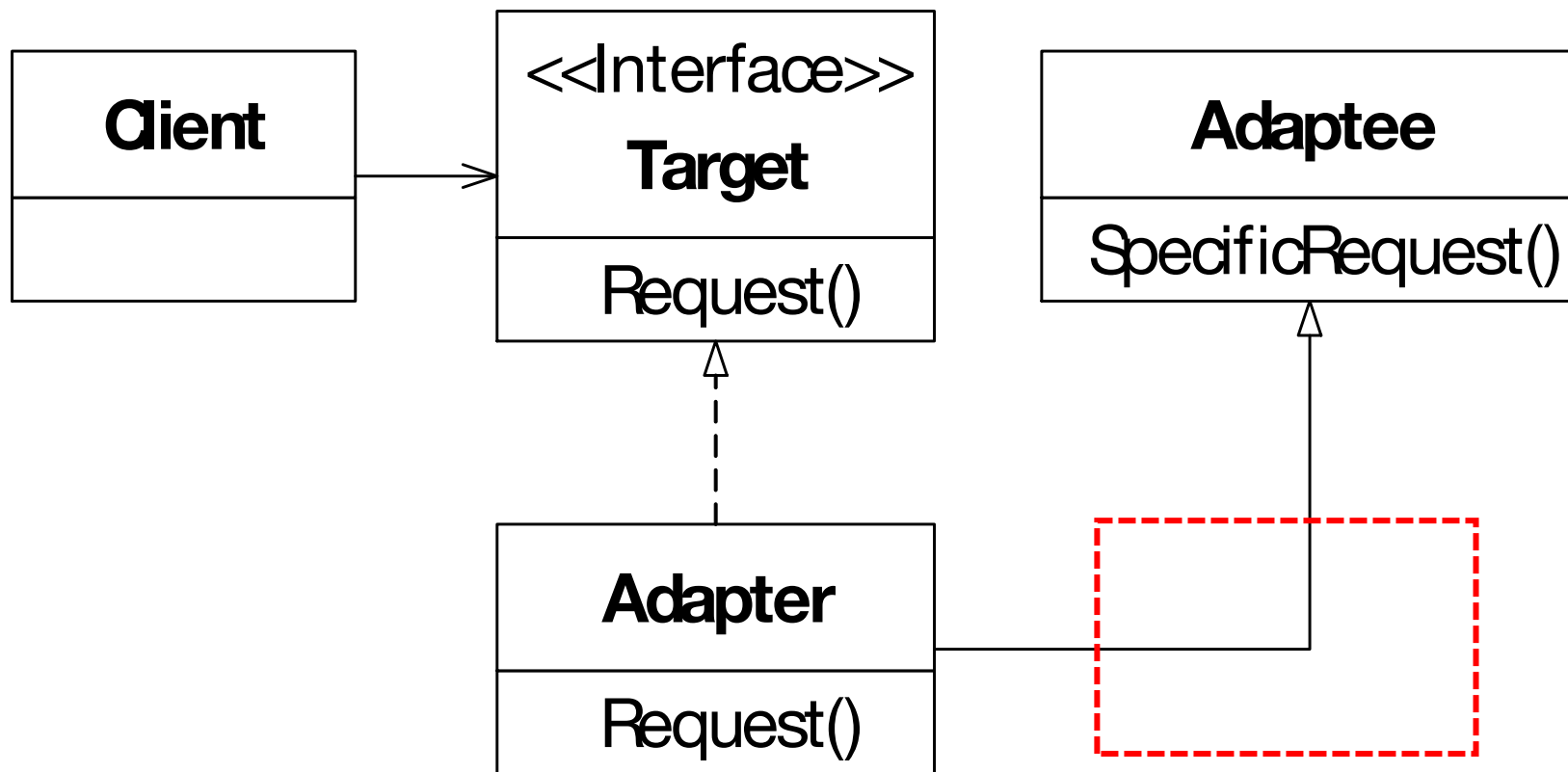
适配器——实现一

```
int main(int argc, char *argv[]) {  
  
    Vector2Stack stack(10);  
  
    //压入1,2,3,4  
    for (int i = 1; i < 5; i++)  
        stack.push(i);  
  
    //逐个弹出  
    for (int i = 0; i < 4; i++) {  
        std::cout << stack.top() << "\n";  
        stack.pop();  
    }  
  
    return 0;  
}
```



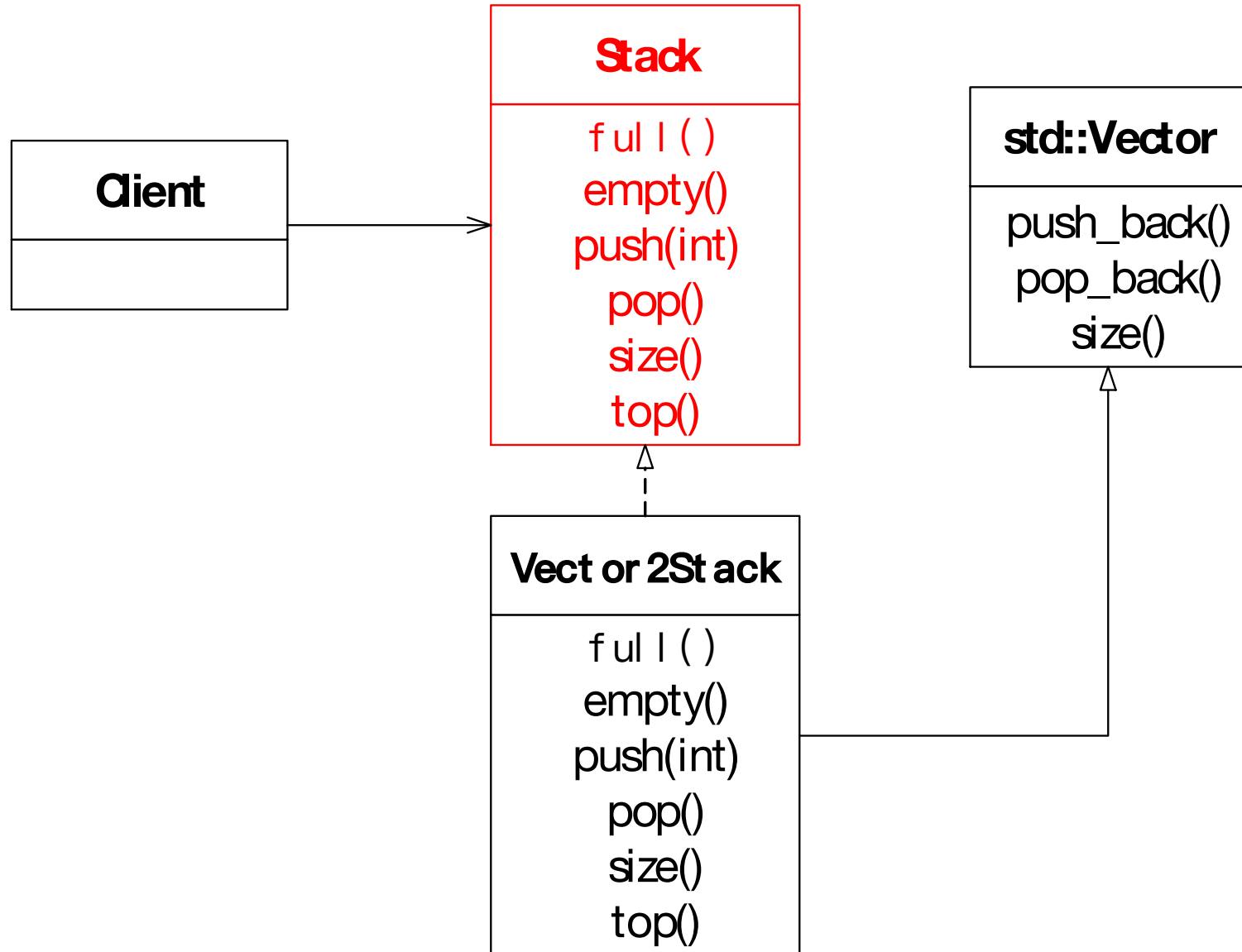
4
3
2
1

适配器——实现二



使用继承实现适配，称作类适配器模式

适配器接口定义

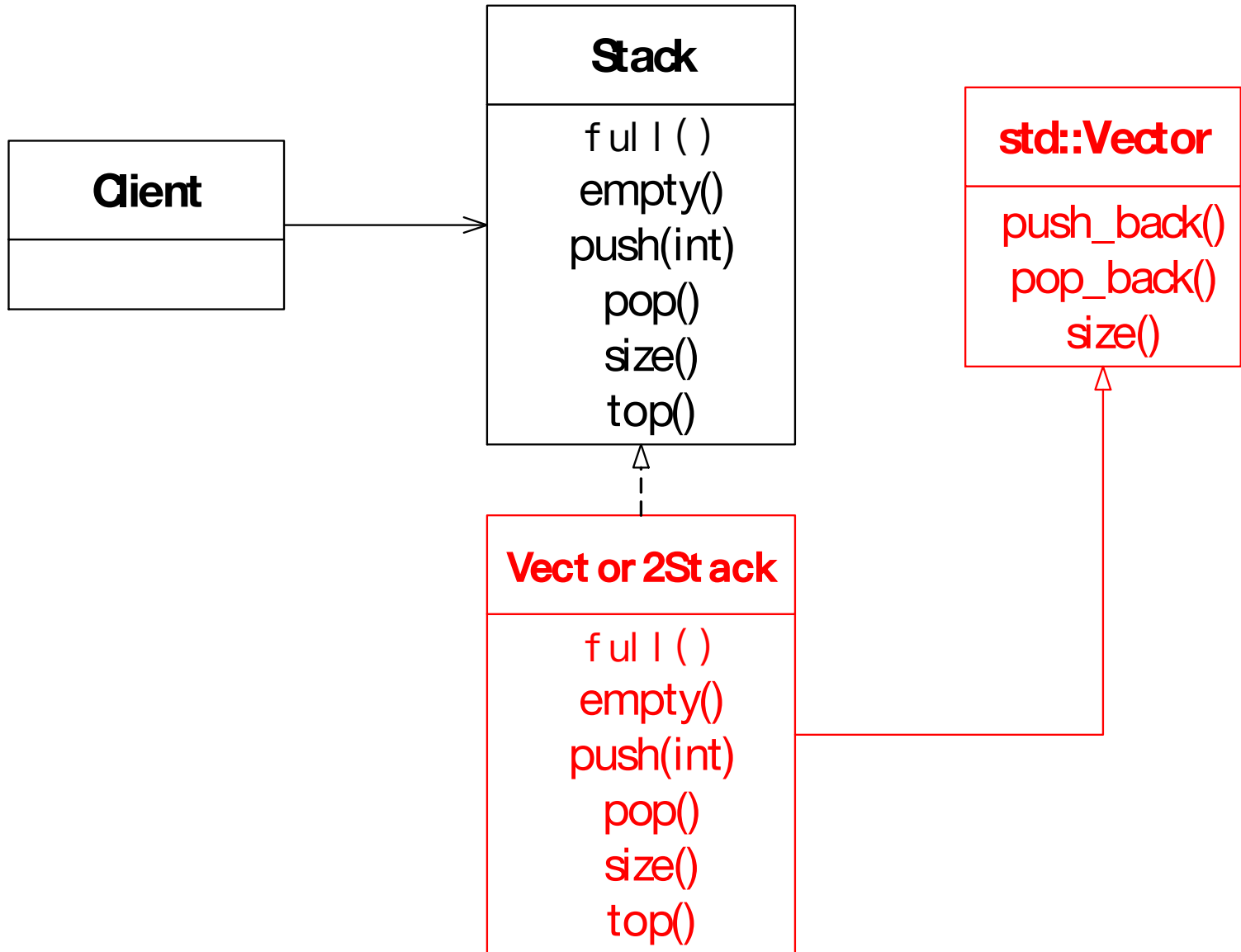


适配器——实现二

```
#include <climits>
#include <vector>
#include <iostream>
using namespace std;

//堆栈基类
class Stack {
public:
    virtual ~Stack() { }
    virtual bool full() = 0;
    virtual bool empty() = 0;
    virtual void push(int i) = 0;
    virtual void pop() = 0;
    virtual int size() = 0;
    virtual int top() = 0;
};
```

继承方式实现适配器模式



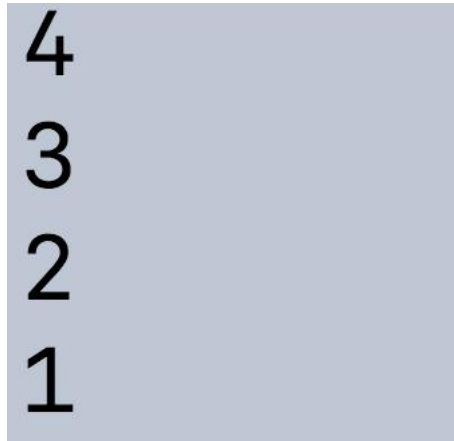
适配器——实现二

// 直接继承vector并改造接口，采用私有继承可以使得外界只能接触到Vector2Stack中的接口

```
class Vector2Stack : private std::vector<int>, public Stack {  
private:  
    int m_size;  
public:  
    Vector2Stack(int size) : m_size(size) {reserve(size);}  
    bool full() { return false; }  
    bool empty() { return vector<int>::empty(); }  
    void push(int i) { push_back(i); }  
    void pop() { pop_back(); }  
    int size() { return vector<int>::size(); }  
    int top() { return back(); }  
};
```

适配器——实现二

```
int main(int argc, char *argv[]) {  
  
    Vector2Stack stack(10);  
  
    //压入1,2,3,4  
    for (int i = 1; i < 5; i++)  
        stack.push(i);  
  
    //逐个弹出  
    for (int i = 0; i < 4; i++) {  
        std::cout << stack.top() << "\n";  
        stack.pop();  
    }  
  
    return 0;  
}
```



4
3
2
1

适配器

■ 优点

- 通过适配器，客户端可以用统一接口调用各种复杂的底层工作类
- 复用了现有的类，提高代码复用率
- 将目标类和适配者类解耦，通过引入一个适配器类包装现有的适配者类以满足新接口需求，无需修改原有代码

■ 适用场景举例

- 系统需要复用已有的类，但这些类的接口不符合系统的接口
- 接入第三方组件，但组件接口定义与自身定义不同
- 旧系统开发的类已经实现了一些功能，但是客户端只能以新接口的形式访问，且我们不希望手动更改原有类

智能指针

- C++中，指针是一个使用起来需要格外注意的东西，尤其是class中有指针，那么析构与释放就会是一个及其棘手的事情，稍有不慎就会程序错误或者内存泄漏。
- 我们要实现智能指针类，能够包裹指针，具有指针的各项功能，并能够进行引用计数，在计数为0时自动释放指针空间。

智能指针

- 使用适配器模式可以进行指针的封装，并对外提供指针各项功能的接口。
- 但是，适配器模式仅仅只是接口的转换，其本身无法在提供接口的同时进行计数这样的功能控制。
- 我们如何在提供功能的同时进行计数控制呢？

代理/委托 Proxy

代理/委托

- 在一些应用中，直接访问对象往往会带来诸多问题
 - 要访问的对象在远程的机器上。
 - 被访问对象创建开销很大，或者某些操作需要安全控制，或者需要进程外的访问
 - 直接访问会给使用者或者系统结构带来不必要的麻烦
 - 被访问对象需要实时根据访问者的行为做出诸多复杂处理
- 我们可以在被访问对象上加~~上~~一个访问层，将复杂操作包裹在内部不对外部类开放，仅对外开放功能接口，即可完成上述要求，这就是代理/委托模式

场景

■ 远程代理：

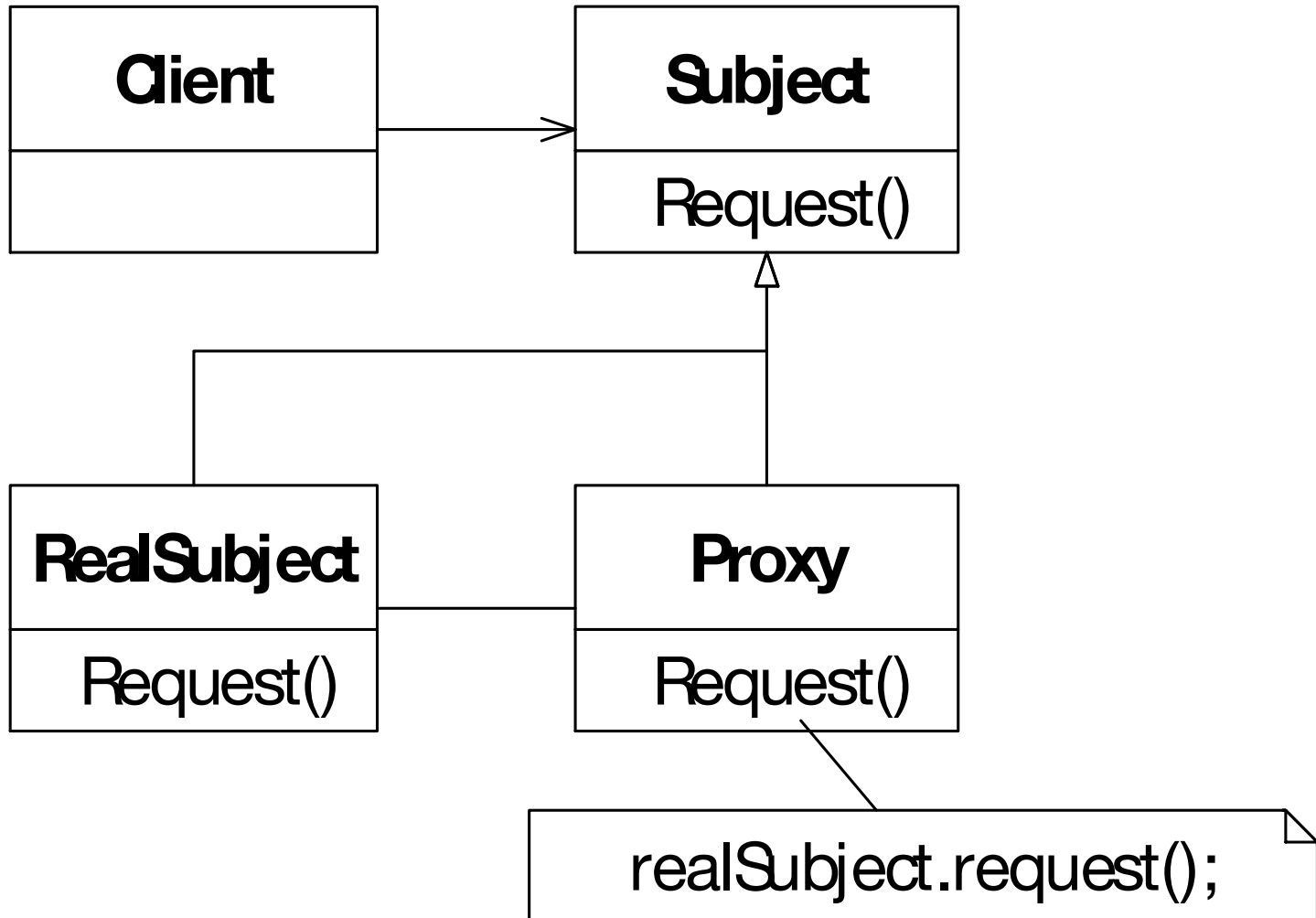
- 在实际应用中，经常会有需要从其他进程或者远程地址获取资源的情况，获取这些资源往往需要较长的时间或者无法直接获取，此时我们即可以使用代理/委托的模式
- 我们利用代理/委托来替换掉原有的获取资源的方式，这样能够更加高效、安全地获取远程资源

■ 资源安全：

- 在多进程编程中，并不是所有进程都对所有资源拥有访问使用修改的权限，所以在进程调用中，我们可以使用代理/委托的模式来检查当前进程对当前资源是否拥有使用权限。

```
void Proxy::request() {  
    if (checkAuthority(nowProcess,nowResource)) {  
        //do something;  
    }  
}
```

代理/委托



例子：智能指针引用计数

```
#include <iostream>
using namespace std;

template <typename T>
//提前声明智能指针模板类
class SmartPtr;

//辅助指针，用于存储指针计数以及封装实际指针地址
template <typename T>
class U_Ptr {
private:
    friend class SmartPtr<T>;

    U_Ptr(T *ptr) :p(ptr), count(1) { }
    ~U_Ptr() { delete p; }

    int count;
    T *p; //数据存放地址
};
```

例子：智能指针引用计数

```
template <typename T>
//智能指针
class SmartPtr {
private:
    U_Ptr<T> *rp; //进行实际指针操作的辅助指针
public:
    SmartPtr(T *ptr) :rp(new U_Ptr<T>(ptr)) { }
    //调动拷贝构造即增加引用计数
    SmartPtr(const SmartPtr<T> &sp) :rp(sp.rp) { ++rp->count; }
    SmartPtr& operator=(const SmartPtr<T>& rhs) {
        ++rhs.rp->count; //赋值号后的指针引用加1
        if (--rp->count == 0) delete rp; //原内部指针引用减1
        rp = rhs.rp; //代理新的指针
        return *this;
    }
    ~SmartPtr() { //只有引用次数为0才会释放
        if (--rp->count == 0) delete rp;
    }
    //对智能指针操作等同于对内部辅助指针操作
    T & operator*() { return *(rp->p); }
    T* operator ->() { return rp->p; }
};
```

例子：智能指针引用计数

```
int main(int argc, char *argv[]) {  
    //声明指针  
    int *i = new int(2);  
    //使用代理来包裹指针  
    SmartPtr<int> ptr1(i);  
    SmartPtr<int> ptr2(ptr1);  
    SmartPtr<int> ptr3 = ptr2;  
    //之后的操作均通过代理进行  
    cout << *ptr1 << endl;  
    *ptr1 = 20;  
    cout << *ptr2 << endl;  
    return 0;  
}
```



2
20

“变”与“不变”

■ SmartPtr<int>与 int* 有相同的接口

- 操作符：*和->
- 赋值操作符与初始化（拷贝构造）
- 释放（析构）

■ SmartPtr<int>比 int* 增加了一些控制操作

- 拷贝构造时引用计数加一
- 析构时引用计数减一，直到引用计数为0时释放
- 赋值时对当前引用计数和参数引用计数分别处理

■ 常被称为“代理”模式

- 接口不变，功能变化
- 用于对被代理对象进行控制，如引用计数控制、权限控制、远程代理、延迟初始化等等
- 代理类就好比被代理类的“经纪人”，一方面提供被代理类所有接口的功能，另一方面可以同时进行额外的控制操作。

代理/委托 与 适配器

■相似：

- 均是在被访问对象之上进行封装
- 均提供被封装对象的功能接口供外部使用

■不同：

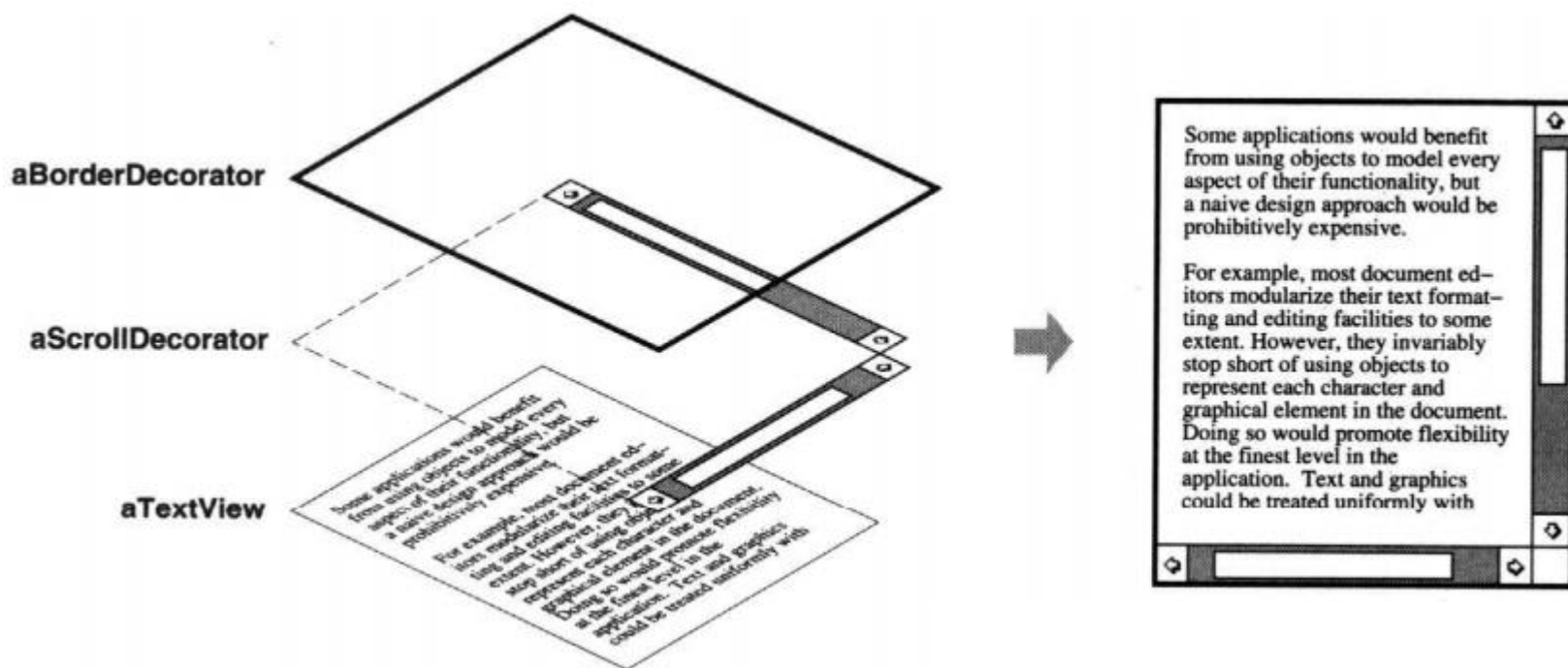
- 代理不会改变接口，但适配器可能会
- 适配器不会增加控制，代理可能会
- 适配器的核心要素是变换接口，代理的核心要素是分割访问对象与被访问对象以减少耦合，并能在中间增加各种控制功能。

装饰器

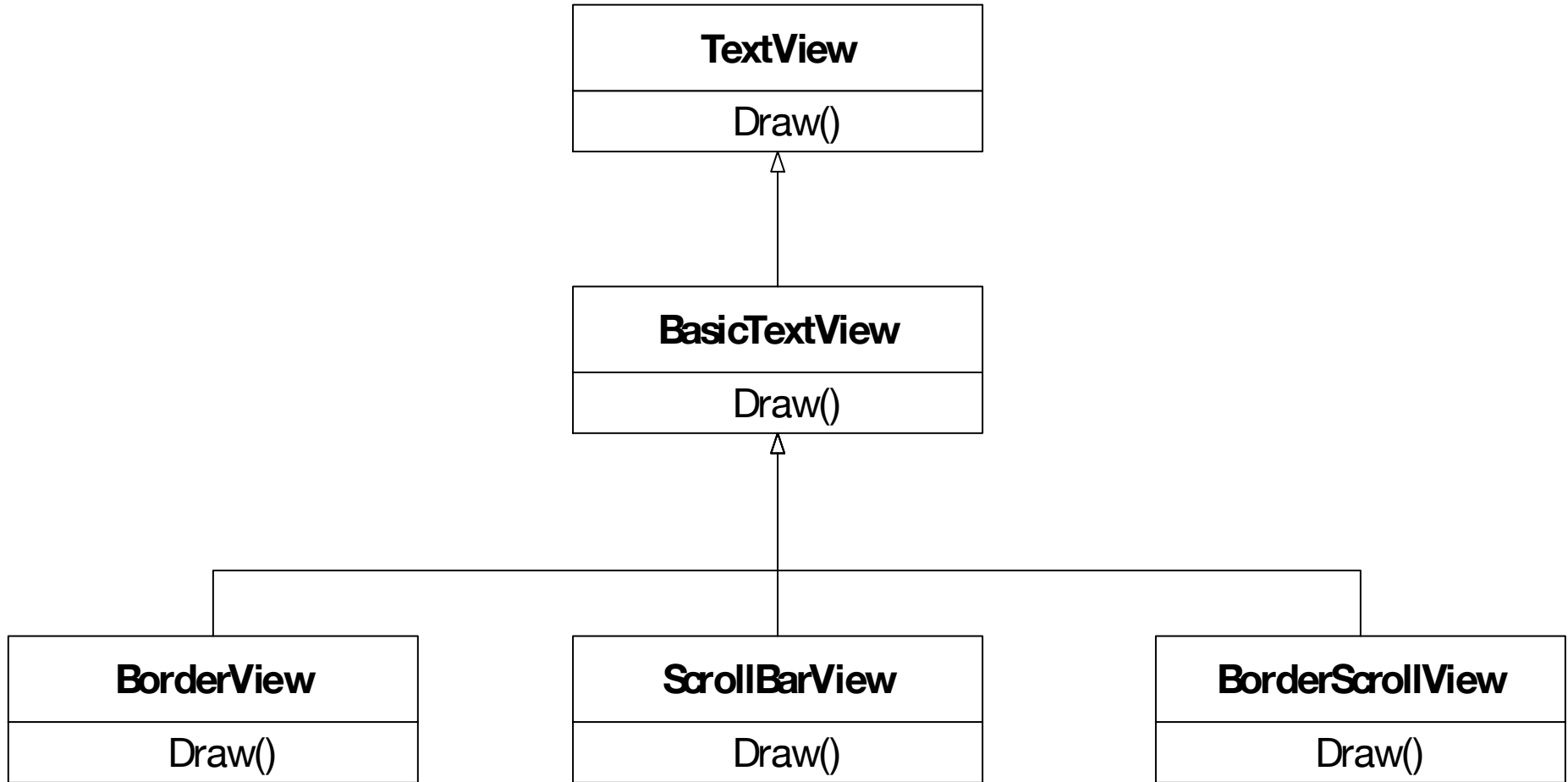
Decorator

例子

- 有一个对象 **TextView**，在窗口中显示文本
- 希望接口不变，增加滚动条、边框、.....



继承



继承

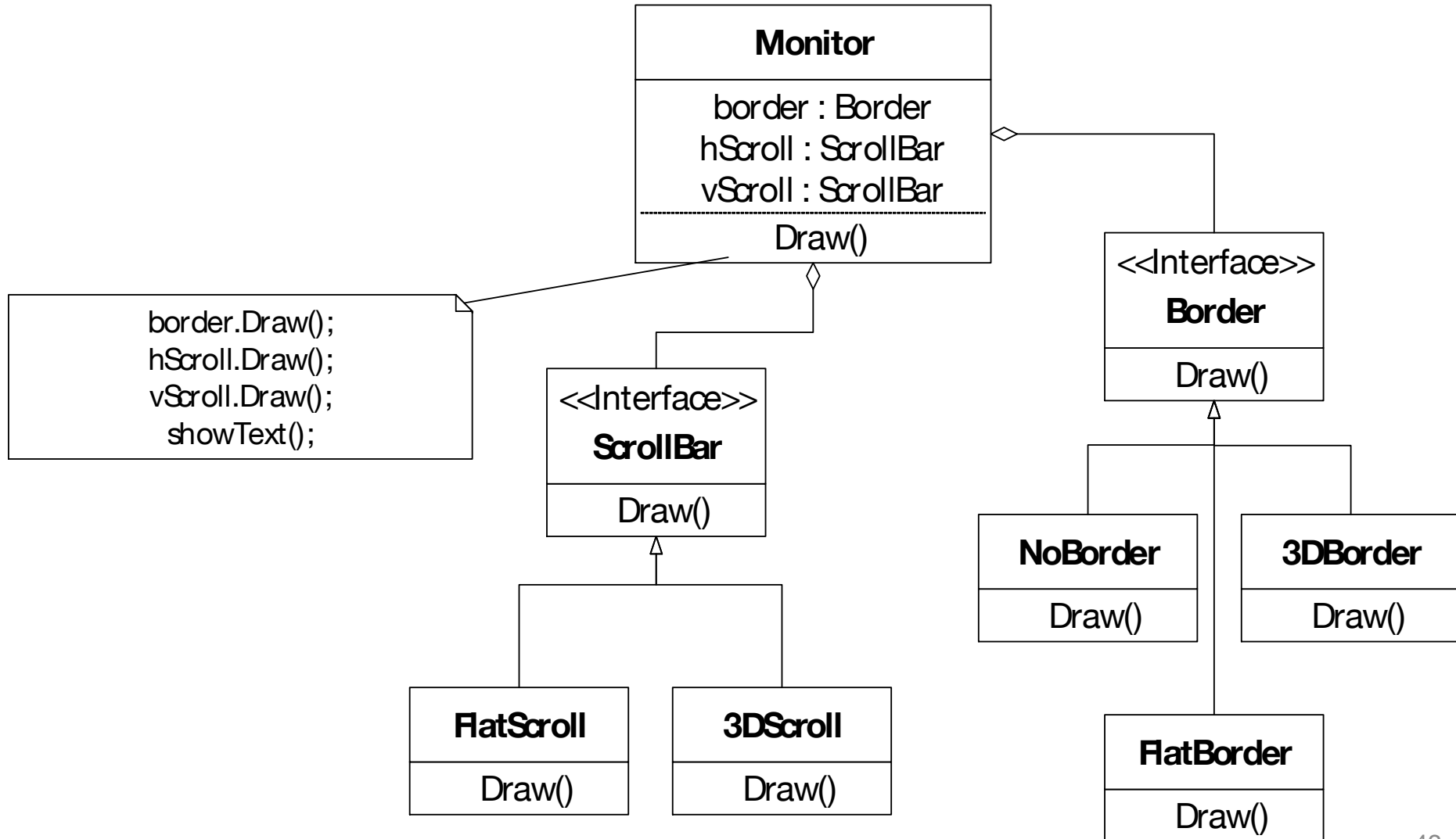
■ 使用继承

■ 依靠多态实现功能的变化

■ 问题

- 随着功能的变多，继承类的数量急剧膨胀，其最大派生类的数目可以是所有功能的组合数
- 如果TextView的基类增加新的接口，那么所有的派生类都需要进行修改

策略



策略

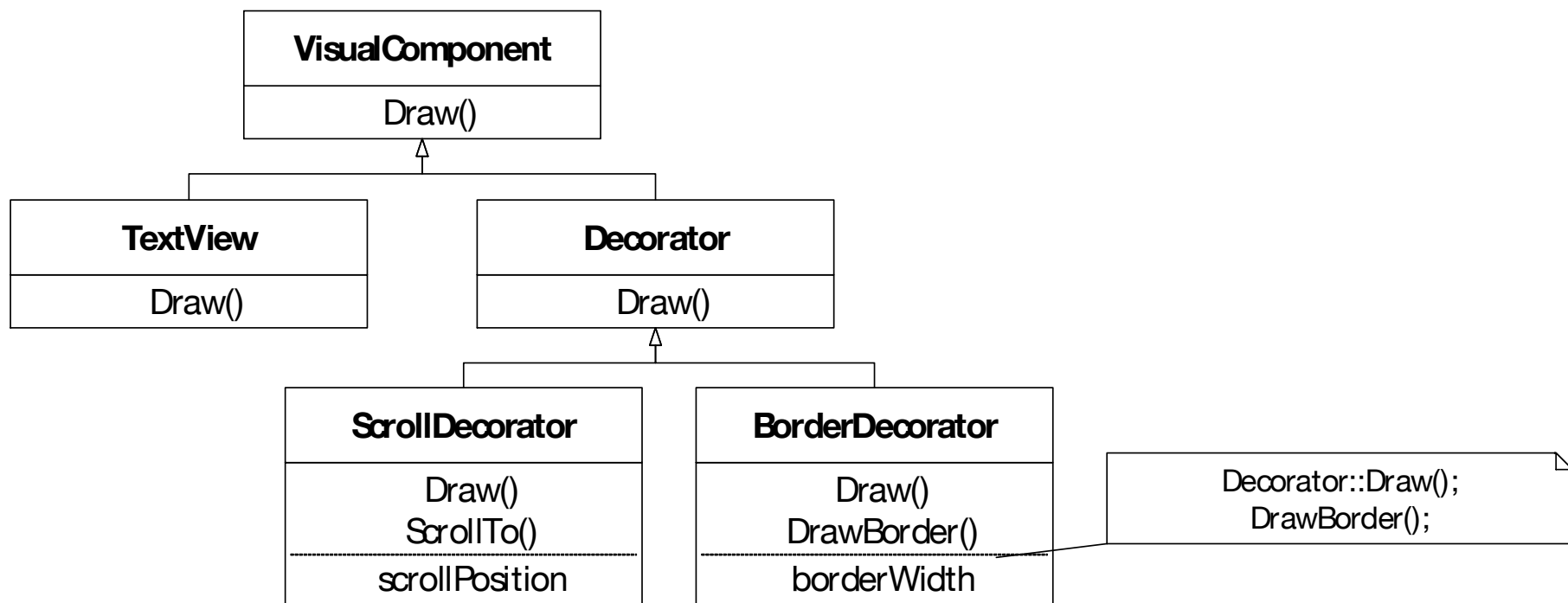
■ 用组合替代继承

■ 问题

- 策略的个数是基类中预先定义好的，如基类中定义了边框和滑动条，那么策略模式只能实现不同的边框与滑动条功能的组合。
- 如果我要再增加一个滚动条和边框之外的新功能，那么就要修改基类，在基类中增加策略个数和新的方法。这样对整体框架的改动是我们不乐意见到的。

装饰器

- 创建了一个装饰类，用来包装原有的类，并在保持接口完整性的前提下，提供了额外的功能。
- 且装饰类与被包装的类继承于同一基类，这样装饰之后的类可以被再次包装并赋予更多功能。



装饰器示例

```
#include <iostream>
using namespace std;

//所有View的基类
class Component {
public:
    virtual ~Component() { }
    virtual void draw() = 0;
};

//一个基本的TextView类
class TextView : public Component {
public:
    void draw() {
        cout << "TextView." << endl;
    }
};
```

装饰器示例

//装饰器的核心内涵在于用装饰器类整体包裹改动之前的类，以保留原来的全部接口

//在原来接口保留的基础上进行新功能扩充

```
class Decorator : public Component {  
    //这里一个基类指针可以让Decorator能够以递归的形式不断增加新功能  
    Component* _component;  
public:  
    Decorator(Component* component) : _component(component) {  
    }  
    virtual void addon() = 0;  
    void draw() {  
        addon();  
        _component -> draw();  
    }  
};
```

代码

```
//包裹原Component并扩充边框
class Border : public Decorator {
public:
    Border(Component* component) : Decorator(component) { }
    void addon() { cout << "Bordered "; }
};
```

```
//包裹原Component并扩充水平滚动条
class HScroll : public Decorator {
public:
    HScroll(Component* component): Decorator(component) { }
    void addon() { cout << "HScrolled "; }
};
```

```
//包裹原Component并扩充垂直滚动条
class VScroll : public Decorator {
public:
    VScroll(Component* component): Decorator(component) { }
    void addon() { cout << "VScrolled "; }
};
```

代码

```
int main(int argc, char** argv) {  
    // 基础的textView  
    TextView textView;  
    // 在基础textView上增加滚动条  
    VScroll vs_TextView(&textView);  
    // 在增加垂直滚动条的基础上增加水平滚动条  
    HScroll hs_vs_TextView(&vs_TextView);  
    // 在增加水平与垂直滚动条之后增加边框  
    Border b_hs_vs_TextView(&hs_vs_TextView);  
    b_hs_vs_TextView.draw();  
    return 0;  
}
```

运行过程与结果

Bordered HScrolled VScrolled TextView.

```
b_hs_vs_TextView.draw();
```

```
Border::addon();  
hs_vs_TextView.draw();
```

Bordered

```
HScroll::addon();  
vs_TextView.draw();
```

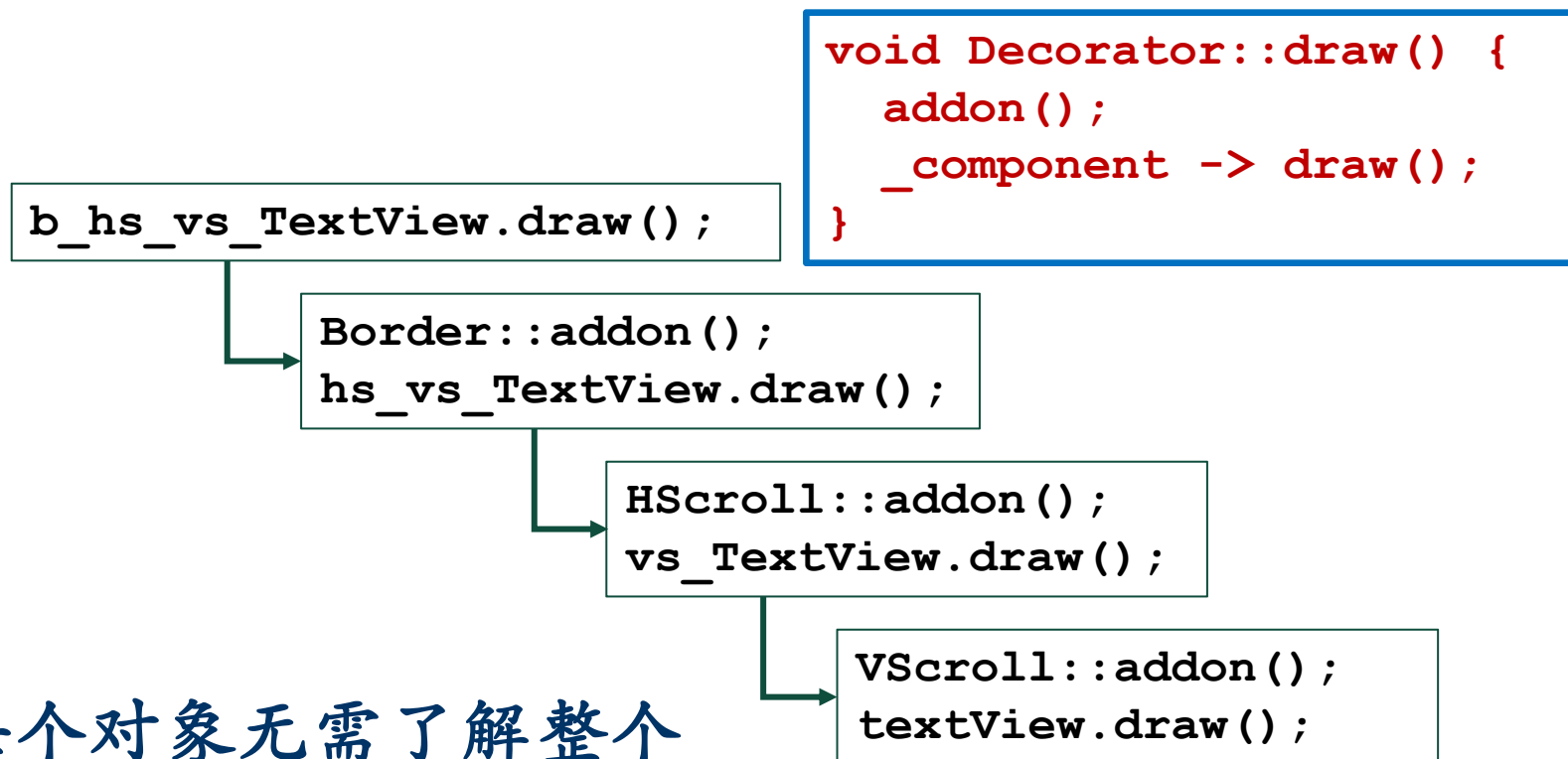
HScrolled

```
VScroll::addon();  
textView.draw();
```

VScrolled

TextView.

调用的链式关系



- 每个对象无需了解整个链的全貌
- 每一次都是将之前的版本完全包裹住，再增加新的功能。换句话说，有多少个新功能就包裹几次

装饰与策略

■ 相同点

- 通过对象的组合修改对象的功能
- 以组合替代简单继承，更加灵活，减少冗余

■ 不同点

策略

- 修改对象功能的内核（行为）
- 组件必须了解有哪些需要选择的策略，侧重于功能选择

装饰

- 修改对象功能的外壳（结构）
- 组件无需了解有哪些可以装饰的内容，侧重于功能组装

装饰与代理

- 都用来改变对象的行为
- 可以把“装饰”看成是一连串的“代理”

装饰

- 为被装饰对象增加额外的行为
- 不影响被装饰对象的原有功能
- 不创建被装饰对象，只是将新功能添加到已有对象上
- 经常多重嵌套装饰

代理

- 常用来对被代理对象进行更精细的控制
- 被代理对象不存在时常创建被代理对象
- 少见多重嵌套

结构型模式

- 结构型设计模式关心对象组成结构上的抽象，包括接口，层次，对象组合等。
 - 适配器模式在类与类之间进行转接，能够提高类的复用度与灵活性
 - 代理/委托模式减少了类与类层次间的耦合，使得类各自的职责清晰
 - 装饰器模式可以动态扩展被装饰类的功能，并留有接口进行持续扩展
- 核心就在于抽象结构层次上的不变量，尽可能减少类与类之间的联系与耦合，从而能够以最小的代价支持新功能的增加。

结束